Deriving Verified Abstract Machines

From Big-Step Semantics in Agda

Mahmoud M. Hamido

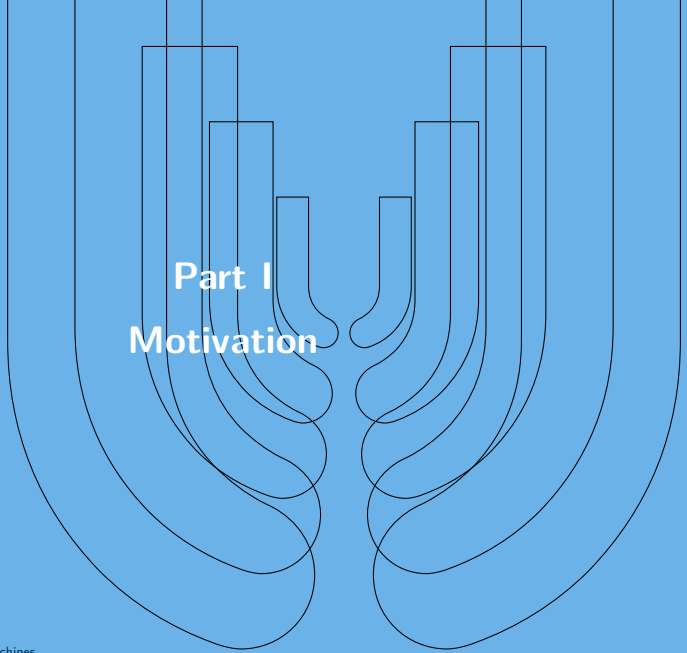
Department of Computer Science
RPTU Kaiserslautern-Landau

May 2026

RPTU Rheinland-Pfälzische Technische Universität Kaiserslautern Landau

Outline

1. Motivation
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work & Conclusion



Part I
Motivation

The Problem: Specs vs. Implementations

The Expectation

A language implementation should faithfully realise its specification.

In practice, specifications are **written documents**:

- C++ Standard
- Java Language Specification
- WebAssembly Specification

Conformance is checked by testing against reference implementations.

The Fundamental Flaw

Testing covers **finitely many** programs.

Specifications make claims about **all** programs.

No matter how thorough the tests, some combination of features may expose a discrepancy.

Formal Verification & This Thesis

The Solution

Prove by construction that an implementation conforms to the specification, covering *all* inputs simultaneously.

Tool: Agda — a dependently typed proof assistant

Foundation: Curry–Howard correspondence

$$\begin{aligned} \text{proposition} &\longleftrightarrow \text{type} \\ \text{proof} &\longleftrightarrow \text{value} \end{aligned}$$

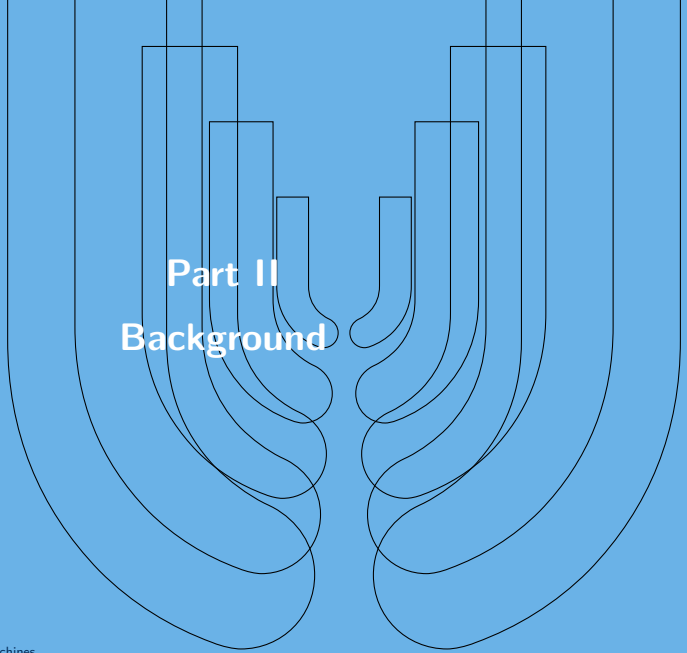
The type checker *is* the proof checker.

Central Thesis

An implementation can be *derived directly* from its specification.

Three case studies:

1. Arithmetic expressions
2. Arithmetic with exceptions
3. Simply Typed Lambda Calculus



Part II

Background

Agda: Dependent Types and Propositions as Types

Types may **depend on values** — invariants are encoded in types.

```
data Vec (A : Set) : N -> Set where
  []  : Vec A zero
  _::_ : A -> Vec A n -> Vec A (suc n)
```

Length tracked at the type level.

Ill-typed terms are not representable.

```
data _==_ {A : Set}(x : A) : A -> Set where
  refl : x == x

-- A well-typed program is a valid proof
+-comm : forall n m -> n + m == m + n
```

Agda checks termination to prevent circular proofs.

Intrinsic Typing

Expressions are **indexed by their type**. Every constructible term is automatically well-typed.

```
data Type : Set where
  Nat  : Type
  Bool : Type

-- Value domain
|= Nat  = N
|= Bool = B
```

```
-- Expressions indexed by Type
data |-_ : Type -> Set where
  `_             : |= T -> |- T
  _+_            : |- Nat -> |- Nat
                 -> |- Nat
  if_then_else_ : |- Bool -> |- T
                 -> |- T -> |- T
```

The condition *must* be a Bool expression.

Both branches *must* share the same type.

The Four Semantics

Style	Describes	Notation	Character
Big-step (natural)	e evaluates to v directly	$e \Downarrow v$	Concise
Small-step (operational)	One-step reductions	$e \longrightarrow e'$	Verbose, fine-grained
Denotational	Maps terms to a meaning domain	$[\![e]\!] = v$	Interpreter-like
Abstract machine	Explicit states + transitions	$s \Rightarrow s'$	Operational

Key Property

Under consistent semantics, they can be **related** — and in some cases one can be *derived* from another.

Proof Strategy: Chain of Equivalences

Rather than proving each pair of semantics equivalent in isolation:

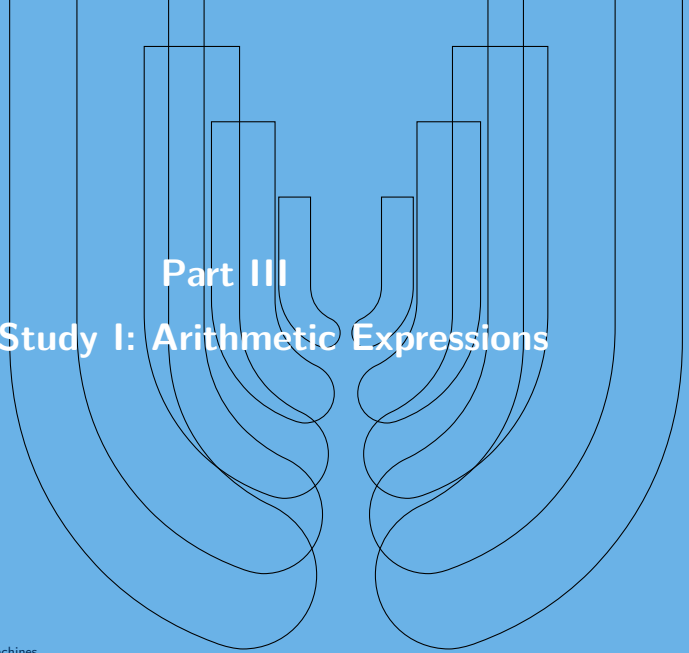
$$\text{Denotational} \longleftrightarrow \text{Big-Step} \longleftrightarrow \text{Machine} \longleftrightarrow \text{Small-Step}$$

- Each link is proved **independently**
- **Transitivity** gives equivalence between any two points
- An indirect equivalence can be composed from intermediate links

Analogy: Intermediate Representations in a Compiler

Rather than translating source directly to assembly in one step, incremental transformations via intermediate representations are used.

Central focus: the link Big-Step $\longleftrightarrow$ Machine



Part III
Case Study I: Arithmetic Expressions

Big-Step Semantics

Denotational ↔ Big-Step ↔ **Machine** ↔ Small-Step

```
data _|>_ : |- T -> |= T -> Set where
  `_      : (v : |= T) -> (` v) |> v

  _+_     : e1 |> n1 -> e2 |> n2
          -> (e1 + e2) |> (n1 + n2)

  if-then : e |> true  -> e1 |> v
          -> (if e then e1 else e2) |> v

  if-else : e |> false -> e2 |> v
          -> (if e then e1 else e2) |> v
```

Key Properties

Deterministic:

$e \Downarrow v_1 \ \wedge \ e \Downarrow v_2$

$\Rightarrow \ v_1 = v_2$

Total:

$\forall e.\ \exists v.\ e \Downarrow v$

Both proved by structural induction on the derivation / expression.

From Interpreter to Abstract Machine

Consider an interpreter as a recursive function traversing the expression tree.

Key Idea: Reify Suspended Computations as *Frames*

For $e_1 \oplus e_2$, the interpreter must:

1. Evaluate e_1 — the whole expression is *suspended* while waiting
   $\Rightarrow$ push frame $(\circ \oplus e_2)$
2. Once v_1 is known, the frame becomes $(v_1 \oplus \circ)$, awaiting e_2

The collection of outstanding frames forms a **stack**.

The machine halts when it reaches a return state with an **empty stack**.

Two machine modes:

Eval $\uparrow e$ evaluating expression e

Return $\downarrow v$ returning value v

Machine: Frames, States, Transitions

```
data Frame : Type -> Type -> Set where
  _+0_ : Hole -> |- Nat -> Frame Nat Nat
  _+1_ : |= Nat -> Hole -> Frame Nat Nat
  _else_ : |- T -> |- T -> Frame T Bool

data State (A : Type) : Set where
  eval   : Stack A T -> |- T -> State A
  return : Stack A T -> |= T -> State A
```

Selected transitions:

```
-- Decompose addition
step-+1 :
  eval stk (e1 + e2)
  ~> eval (stk :: (+0 e2)) e1

-- Left known; evaluate right
step-+2 :
  return (stk :: (+0 e2)) v1
  ~> eval (stk :: (v1 +1)) e2

-- Both known; sum and return
step-+3 :
  return (stk :: (v1 +1)) v2
  ~> return stk (v1 + v2)
```

Correctness and Completeness

Completeness: big-step ⇒ machine

```
complete : e |> v -> (eval stk e) ~>* (return stk v)
```

> Proved by **induction on the big-step derivation**

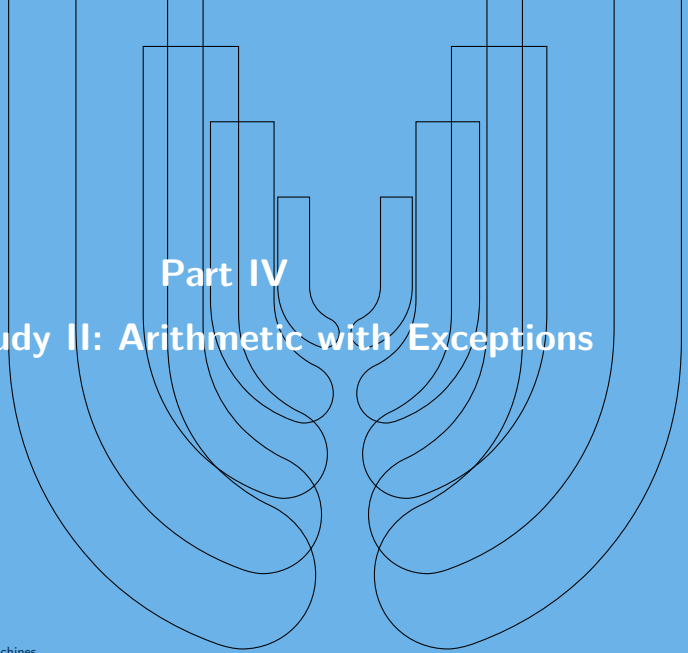
> Must generalise over an *arbitrary* stack

Correctness: machine ⇒ big-step (via confluence)

```
correct : (eval [] e) ~>* (return [] v) -> e |> v
```

1. **Totality** gives a canonical big-step derivation to some v'
2. **Completeness** simulates it into a machine run ending at $\downarrow v'$
3. **Confluence**: two runs from e to values must agree $\Rightarrow v = v'$

This confluence argument is **reused in all three case studies**.



Part IV
Case Study II: Arithmetic with Exceptions

Extending the Language with Exceptions

```
data Exception : Set where
  err : Exception

-- New expression forms
throw      : Exception -> |- T
try_catch_ : |- T
           -> (Exception -> |- T)
           -> |- T
```

A single `err` constructor suffices.

A tagged exception type allows handlers to re-raise on non-matching tags.

Terms now evaluate to a **result**:

```
data Result (T : Type) : Set where
  return : |= T       -> Result T
  raise  : Exception -> Result T
```

Convenient aliases:

$> \quad e \Downarrow v \triangleq e \Downarrow^{r} \text{return } v$

$> \quad e \Uparrow ex \triangleq e \Downarrow^{r} \text{raise } ex$

Big-Step Semantics with Exceptions

Denotational ↔ Big-Step ↔ **Machine** ↔ Small-Step

Uniform pattern: one rule for *normal return*, one for *exception propagation*.

```
data _|>r_ : |- T -> Result T -> Set where
  _+_     : e1 |> n1 -> e2 |> n2 -> (e1 + e2) |> (n1 + n2)
  +-exn1  : e1 ^ ex               -> (e1 + e2) ^ ex
  +-exn2  : e1 |> n1 -> e2 ^ ex   -> (e1 + e2) ^ ex

  throw-rule : throw ex |>r raise ex

  try-return : e |> v                -> (try e catch h) |> v
  try-catch  : e ^ ex -> h ex |>r r -> (try e catch h) |>r r
```

The handler *h* receives the exception and may itself succeed or raise further.

The Machine: Three Modes and Unwind

A new **unwind mode** carries an exception through the stack:

```
data State (A : Type) : Set where
  eval   : Stack A T -> |- T      -> State A
  return : Stack A T -> |= T      -> State A
  unwind : Stack A T -> Exception -> State A
```

New frame and key transitions:

```
catch : (Exception -> |- T) -> Frame T T

step-throw :
  eval stk (throw ex) ~> unwind stk ex

step-catch-exn :
  unwind (stk :: catch h) ex ~> eval stk (h ex)
```

Per-frame drop rules

Each frame needs an *explicit* drop rule when unwinding:

```
step-drop-else :
  unwind (stk :: (e1 else e2)) ex
  ~> unwind stk ex

step-drop-+0 :
  unwind (stk :: (+0 e2)) ex
  ~> unwind stk ex
```

A single generic drop rule makes the completeness proof impossible: without inspecting the frame we cannot determine which big-step constructor applies.

Mutual Recursion in Completeness

```
complete     : e |> v  -> (eval stk e) ~>* (return stk v)
complete-exn : e ^ ex  -> (eval stk e) ~>* (unwind stk ex)
```

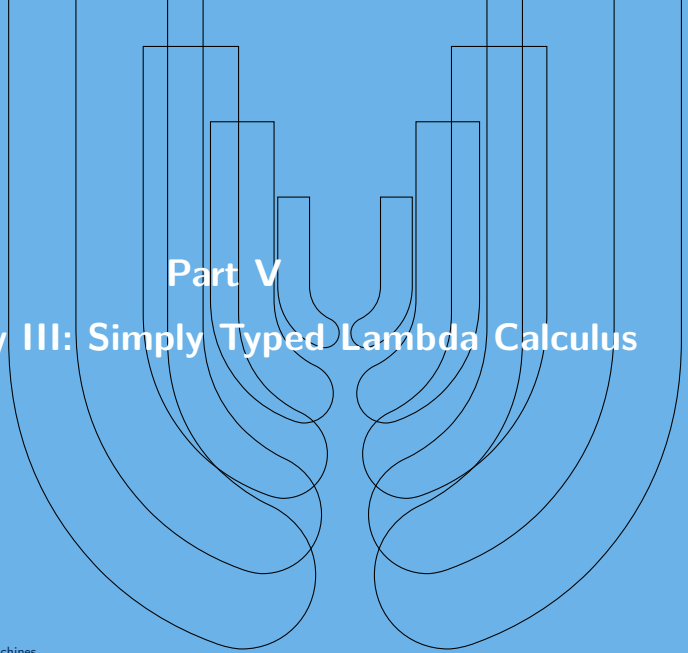
Why mutual recursion?

> The `try-catch` case of `complete` needs `complete-exn` to handle the body (which raised)

> `complete-exn` calls `complete` to handle the handler (which produces a value)

Correctness extended to two terminal states.

If the machine reaches the *wrong* terminal state ($\downarrow$ vs. unwind), an absurd equality between distinct constructors is discharged by `ex-falso`.



Part V
Case Study III: Simply Typed Lambda Calculus

Syntax and De Bruijn Indices

```
data _|-_ (G : Context) : Type -> Set where
  Nat  : N -> G |- Nat
  Bool : B -> G |- Bool
  _+_  : G |- Nat -> G |- Nat -> G |- Nat
  Var  : T in G -> G |- T
  lam  : (G , T1) |- T2 -> G |- (T1 => T2)
  _·_  : G |- (T1 => T2) -> G |- T1 -> G |- T2
```

De Bruijn indices

Variables as *membership proofs* in the typing context:

```
data _in_ : Type -> Context -> Set where
  Z : T in (G , T)
  S : T in G -> T in (G , U)
```

Identity $\lambda x.x$ $\lambda\, Z$

Constant $\lambda x.\lambda y.x$ $\lambda\,\lambda\,(S\ Z)$

Scope safety is guaranteed by the type.

Closures and Big-Step Semantics

Denotational $\leftrightarrow$ Big-Step $\leftrightarrow$ **Machine** $\leftrightarrow$ Small-Step

λ-abstractions produce **closures**: a saved environment paired with the function body.

```
record Closure (A B : Type) : Set where
  inductive
  constructor <_,_>
  field
    {Gc} : Context
    dc   : Environment Gc
    ec   : (Gc , A) |- B

|= (A => B) = Closure A B  -- mutually recursive
```

```
data _|-_|>_ :
  Env G -> G |- T -> |= T -> Set where

  VAR : d |- Var x |> lookup d x

  ABS : d |- lam e |> < d , e >

  app : d |- e0 |> < dc , ec >
      -> d |- e1 |> v1
      -> (dc , v1) |- ec |> v
      -> d |- e0 . e1 |> v
```

Totality uses {-# TERMINATING #-}: the recursive call on the closure body is not structurally smaller.

Denotational Semantics: Two Approaches

Direct Semantics

$[\![A \Rightarrow B]\!] = [\![A]\!] \to [\![B]\!]$

```
interpret d (lam e) =
  \z -> interpret (d , z) e

interpret d (e1 . e2) =
  (interpret d e1)(interpret d e2)
```

Function types are Agda functions.

Closure Semantics

$[\![A \Rightarrow B]\!] = \mathsf{Closure}\ A\ B$

```
evaluate d (lam e)    = < d , e >
evaluate d (e1 . e2)  =
  apply (evaluate d e1)(evaluate d e2)

apply < dc , ec > v = evaluate (dc , v) ec
```

Matches the big-step value domain exactly.

The two are related by **logical relations**: $v \sim_T u$ for each type T.

For function types: applying related functions to related arguments produces related results.

STLC Abstract Machine: Environment in State

The environment is **carried in the machine state**: $\delta \vdash \texttt{stk} \uparrow e$.

Three frames for each phase of function application:

```
data Frame : Type -> Type -> Set where
  app1 : Hole -> G |- A    -> Frame B (A => B)
  app2 : |= (A => B) -> Hole -> Frame B A
  app3 : Env G -> |= (A => B) -> |= A
       -> Hole -> Frame B B
```

Three phases of $e_1 \cdot e_2$

1. **app$_1$**: evaluate e_1 to closure $\langle \delta^c, e^c \rangle$
2. **app$_2$**: evaluate e_2 to argument v_2
3. **app$_3$**: switch to δ^c, evaluate body e^c; *restore caller's environment* on return

The Open Problem: Small-Step Equivalence for STLC

The Mismatch

Small-step: substitution

$$(\lambda\, e_1) \cdot v_2 \longrightarrow e_1[v_2/0]$$

Big-step: environment lookup

$$(\delta, v_2) \vdash e_1 \Downarrow v$$

To relate them requires embedding values back as terms:

embed $:\vDash T \to \emptyset \vdash T$

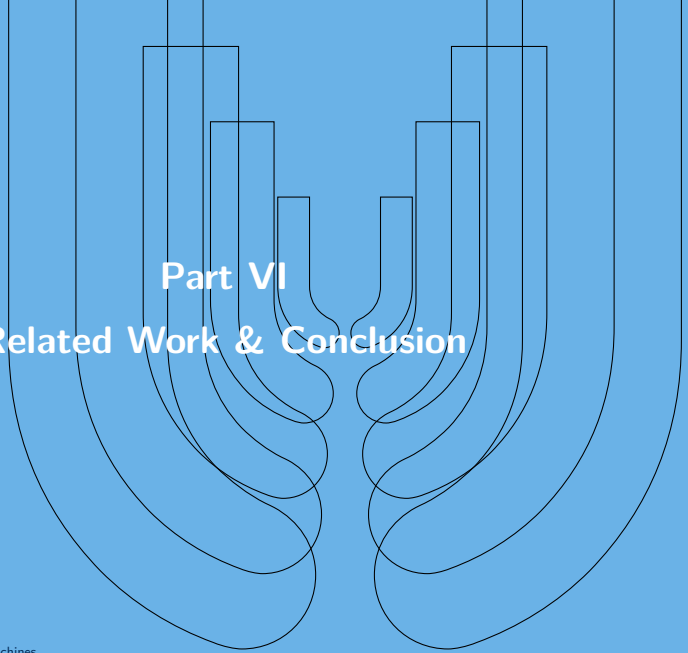
But closures $\langle \delta^c, e^c \rangle$ are defined in context $\Gamma^c \neq \emptyset$ and the captured environment cannot be embedded.

Left open in this thesis.

Possible approaches:

- Kripke logical relations
- Named-variable representation with explicit substitution lemmas
- Unified representation where both semantics share the same notion of value

An alternative formulation ("STLC-Alt") using a `quote` constructor was explored but does not resolve the core issue.



Part VI
Related Work & Conclusion

Related Work

Hutton et al. ("Easy as 1,2,3") Evaluator → CPS → defunctionalization → machine.

A frame in this thesis ≡ a defunctionalized continuation. Correctness by construction; different derivation route.

Hutton & Wright ("Calculating Exceptional Machines") Same CPS approach extended with exceptions.

The continuation type is modified to account for exception-raising vs. returning.

Hutton et al. ("Compiling Exceptions Correctly") Verified compiler to a stack-based machine; handlers pushed as code.

Exceptions unwind until a matching handler is found.

Hutton et al. ("Calculating Dependently Typed Compilers") Agda formalization — *without* explicit correctness proofs.

This Thesis

Derives machines from the *structure of big-step derivations* directly, with **machine-checked** correctness and completeness proofs.

Summary

Three Completed Case Studies

Arithmetic	full chain
Exceptions	full chain + unwind mode
STLC	big-step ↔ machine

For each language:

- Intrinsically typed syntax
- Big-step semantics (total, deterministic)
- Abstract machine derived from frames
- Machine-checked completeness + correctness
- Denotational semantics + equivalence

Semantic equivalence chain:

Denotational ↔ Big-Step ↔ Machine

Uniform proof methodology:

- Completeness: induction on big-step derivation
- Correctness: totality + completeness + confluence
- Same pattern across all three case studies

All Agda code type-checks.

Correctness guaranteed by construction, not by testing.

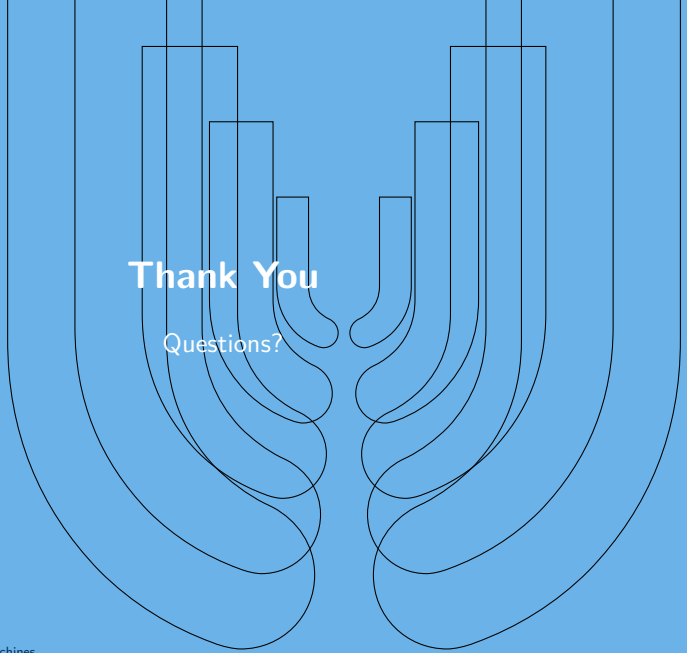
Limitations & Future Work

Current limitations:

1. **Totality assumption**
   Big-step semantics cannot represent non-terminating programs.
2. **Termination pragma in STLC**
   The totality proof is accepted on trust, not verified structurally.
3. **Small-step $\leftrightarrow$ big-step for STLC**
   Substitution vs. environment lookup remains unproven.

Future directions:

- Coinductive big-step or pivot to small-step as the primary specification
- Find a structurally terminating proof for STLC totality
- Prove STLC small-step equivalence via Kripke logical relations
- Abstract the derivation methodology into a reusable framework



Thank You

Questions?